

biba&boba production

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к архитектурному этапу (М2)

курсового проекта

«Survive as a Student»

текстовый roguelite-RPG про выживание первокурсника

Язык реализации: C++20

Платформа: Windows (консольное приложение)

Дата защиты: 20 апреля 2026 г.

Санкт-Петербург, 2026

1. Технологический стек и принципы

1.1. Основной стек

- **Язык:** C++20.
- **Стандартная библиотека:** широко используются контейнеры STL, умные указатели, `std::optional`, `std::variant`, `std::unordered_map`, `std::string_view`.
- **Единственная внешняя зависимость:** библиотека `nlohmann/json` для парсинга и сериализации контентных файлов.
- **Сборка:** Visual Studio 2022 + Clang Power Tools, конфигурация через `CMakeLists.txt`.
- **Формат контента:** JSON. Все события, персонажи, предметы, вопросы экзаменов хранятся в файлах `data/*.json` и подгружаются на старте игры.

1.2. Архитектурные принципы

В проекте сознательно приняты несколько сквозных принципов, которые дисциплинируют разработку и делают код предсказуемым:

1.2.1. Данные — в JSON, логика — в C++

Ни одна сюжетная строчка, ни одно число баланса, ни один шаблон события не захардкожены в C++. Весь контент читается из `data/` при старте игры. Это означает, что любые сценарные правки не требуют перекомпиляции, а в перспективе позволят, например, дать сценаристу (в нашем случае — соавтору проекта) возможность менять события без участия программиста.

1.2.2. Только умные указатели

Ни одного «сырого» указателя (`T*`) в проекте быть не должно. Для владения используется `std::shared_ptr<T>`, для ссылок без владения — `std::weak_ptr<T>`. Это решает сразу несколько проблем: исключает утечки памяти, делает владение явным (кто хранит — владеет), предотвращает висящие ссылки на удалённых NPC или предметы, и, что важно, делает сериализацию/десериализацию мира принципиально возможной (weak-ссылки легко восстановить по ID после загрузки).

1.2.3. Слои с однонаправленными зависимостями

Система разделена на пять слоёв: Engine, Systems, IO, Domain, Content. Зависимости идут строго сверху вниз: Engine знает обо всех остальных слоях, Systems знает только о Domain, IO работает с Domain и Content, Domain не знает ни о ком. Такое разделение гарантирует, что изменения в подсистемах (например, добавление новой системы) не просачиваются в доменные классы, а доменные классы остаются чистыми моделями без «поведенческой» логики.

1.2.4. Данные — в Domain, глаголы — в Systems

Доменные классы (Player, NPC, WorldState и т. д.) хранят состояние и предоставляют геттеры/сеттеры, но не выполняют игровую логику. Например, класс Player не содержит метод «получить урон от бодуна» — за это отвечает StatusEffectSystem. Это классический паттерн anemic domain model, сознательно выбранный ради читаемости: чтобы понять, «что может происходить в игре», достаточно пройтись по методам систем.

1.2.5. Один корневой агрегат — WorldState

Всё состояние партии (кроме совершенно эфемерных данных вроде временных UI-буферов) живёт внутри одного объекта WorldState. Это точка входа и точка выхода для всех систем. Никакие глобальные переменные, никакие синглтоны не используются: WorldState передаётся по ссылке в конструкторы систем и далее циркулирует как параметр в методах.

1.2.6. Нет сохранений

Принципиальное решение: в игре отсутствует механизм save/load. Это заложено в концепцию жанра и упрощает архитектуру — не требуется поддерживать совместимость между версиями, не нужна версионированная сериализация, не возникает классов проблем с «полем появилось после сохранения». Одна партия — одна непрерывная сессия, от новой игры до конца главы 2 (успешного или нет).

2. Общая архитектура

2.1. Пять слоёв

Высокоуровневая карта системы выглядит так:

- **Engine.** Точка входа приложения: GameApp, GameLoop. Инициализирует подсистемы, ведёт главный цикл, маршрутизирует ввод.
- **Systems.** Семь подсистем, реализующих игровую логику: EventSystem, StatusEffectSystem, RelationshipSystem, AcademicSystem, EconomySystem, TimeSystem, EndingSystem.
- **IO.** Ввод-вывод: ConsoleRenderer отрисовывает мир в консоль, InputHandler читает нажатия клавиш, AssetLoader парсит JSON-контент.
- **Domain.** Доменная модель: Player, World (в т. ч. WorldState и Calendar), Characters (NPC, Group, Relationship), Events (Event, Choice, Outcome, Trigger), Academics (Subject, Teacher, Question, CheatSheet, Exam).
- **Content.** Чистые данные: файлы events.json, npcs.json, subjects.json, items.json, dialogues.json в каталоге data/. Никакого кода, только данные.

2.2. Направление зависимостей

Зависимости строго однонаправленные. Engine зависит от всех остальных слоёв. Systems зависит только от Domain. IO зависит от Domain и читает Content. Domain не зависит ни от чего за своими границами. Content вообще не имеет зависимостей — это файлы JSON.

Такая структура позволяет тестировать домен изолированно (без консоли и без систем), писать юнит-тесты для систем, подменяя только WorldState, и, в перспективе, заменять IO-слой (например, на web-версию с WASM) без трогания остальных слоёв. Общая карта компонентов и их связей показана на рис. 1.

Survive as a Student – Component Diagram (M2, block 1, v0.1)

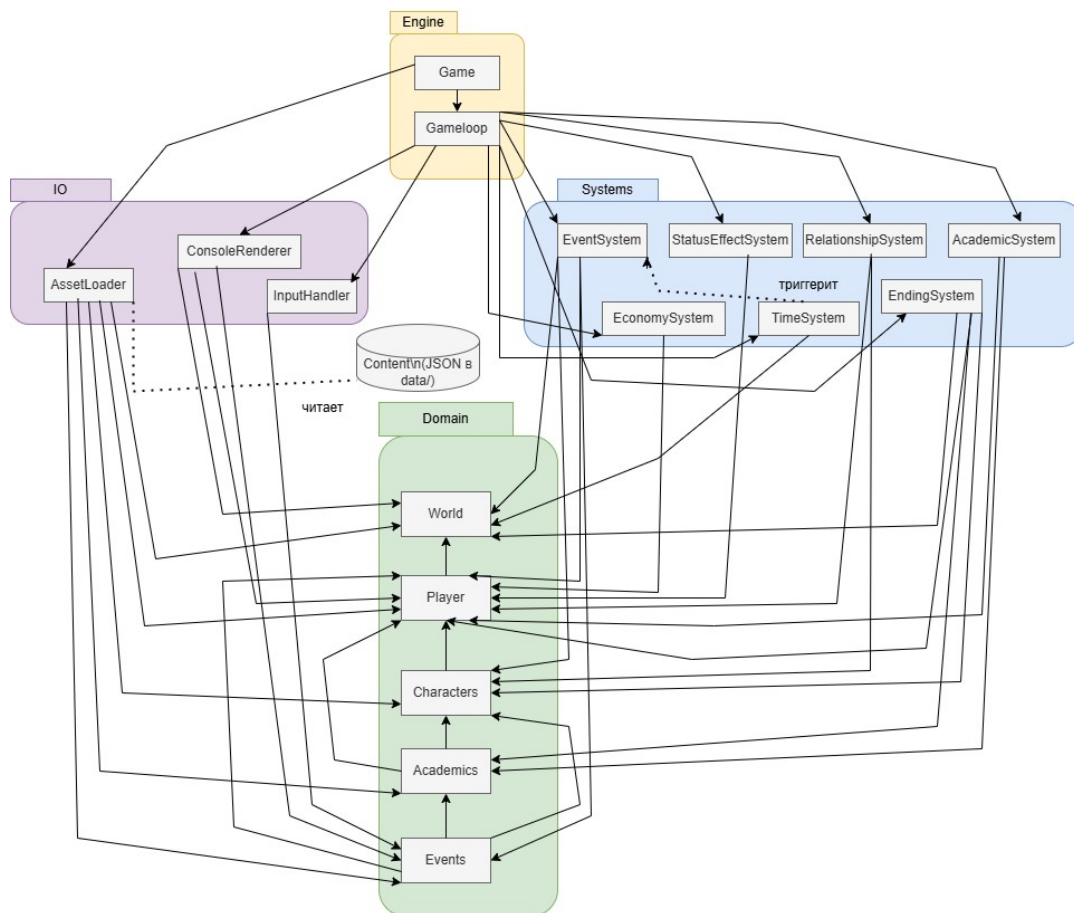


Рис. 1. Компонентная диаграмма системы

2.3. Жизненный цикл объектов

На старте GameApp создаёт Renderer, InputHandler и AssetLoader. AssetLoader читает JSON-файлы, возвращает коллекции доменных объектов (events, npcs, subjects, items). GameApp строит WorldState, наполняя его каталогами из этих коллекций, создаёт Player и помещает в WorldState. Затем конструируются все семь систем, каждая получает ссылку на WorldState. Управление передаётся GameLoop, который прокручивает главный цикл до игры-овера.

3. Ядро: WorldState

3.1. Роль WorldState

WorldState — корневой агрегат, владеющий всем состоянием игры. Его инварианты:

1. На любой момент времени существует ровно один экземпляр WorldState.
2. Player принадлежит WorldState через `std::shared_ptr`.
3. Все каталоги (`npcs`, `locations`, `items`, `teachers`, `events`) — отображения `std::unordered_map<Id, std::shared_ptr<T>>`.
4. Ссылки между доменными объектами (например, с NPC на Group, с Subject на Teacher) — `std::weak_ptr<T>`, чтобы не возникали циклы владения.

3.2. Структура полей

WorldState агрегирует:

- `player` — указатель на единственного игрока.
- `calendar` — объект `Calendar`, ведущий учёт дня, недели, номера текущей главы.
- `events` — каталог шаблонов событий, загруженный из `events.json`.
- `npcs` — каталог всех NPC (важных и фоновых).
- `locations` — каталог локаций (общага, бар, столовая, улица, университет и т. д.).
- `teachers` — каталог преподавателей с их академическими параметрами.
- `items` — каталог шаблонов предметов.
- `journal` — `EventJournal`, хронологический лог выборов игрока.
- `sessionStage` — текущая стадия сессии (`NOT_IN_SESSION` / `SESSION` / `DOPSA` / `KOMSA`).
- `flags` — `std::unordered_map<std::string, bool>`, произвольный набор сюжетных флагов.
- `counters` — `std::unordered_map<std::string, int>`, произвольные счётчики (сколько раз был в баре, сколько раз дрался и т. д.).
- `chapterOutcomes` — коллекция слепков `ChapterOutcome`, фиксирующих состояние на границе глав.

3.3. Метод `validate()`

`WorldState::validate()` проверяет набор инвариантов и возвращает список нарушений (в production-сборке — бросает исключение при первом). Проверяется, в частности:

5. Каждый шаблон события ссылается на существующие NPC и локации.
6. Каждый NPC принадлежит либо ровно одной группе, либо ни одной (`weak`-ссылка либо валидна, либо пуста).
7. Для каждого Subject задан живой Teacher.

8. `Calendar.today` принадлежит текущей главе (не выходит за её диапазон дней).
9. У `Player` все четыре органа инициализированы.
10. `sessionStage` совместима с `Calendar.today.type` (например, `SESSION` допускается только в `SESSION_DAY`).
11. Если задан `Player.aligned` (идеологическая группа), эта группа существует в каталоге.
12. Все ID в каталогах уникальны.

3.4. Календарь и структура глав

Одна партия состоит из двух глав, каждая соответствует одному семестру. Длина главы — 60 игровых дней, из которых первые 10 насыщены сюжетными событиями (`PLOT`), дни 11–45 — смесь `STUDY_ONLY` и `EVENTFUL`, дни 46–60 — сессия (`SESSION_DAY`) и, при необходимости, допса/комса. Последний день главы — `CHAPTER_END`, когда собирается `ChapterOutcome`.

Тип дня определяется перечислением `DayType`: `PLOT`, `STUDY_ONLY`, `EVENTFUL`, `SESSION_DAY`, `CHAPTER_END`. Один и тот же номер дня во второй главе — это другой физический день; календарь после перехода сбрасывается.

3.5. Переход между главами

Переход реализует `TimeSystem::transitionToNextChapter()`. Он не пересоздаёт `WorldState` — это был бы огромный побочный эффект. Вместо этого происходит избирательный сброс: `Calendar` инициализируется заново для главы 2, `sessionStage` сбрасывается в `NOT_IN_SESSION`, флаги, связанные с сессией, обнуляются. Всё, что определяет личность героя — статьи, органы, знания, инвентарь, отношения, — переносится без изменений. `ChapterOutcome` первой главы сохраняется в `WorldState::chapterOutcomes` и становится доступным для сюжетных триггеров второй главы (например, событие может спросить «А ты примкнул к Артёму в первой главе?»).

Структура ядра и отношения между `WorldState`, `Player`, `Calendar` и связанными `value-`объектами представлены на рис. 2.

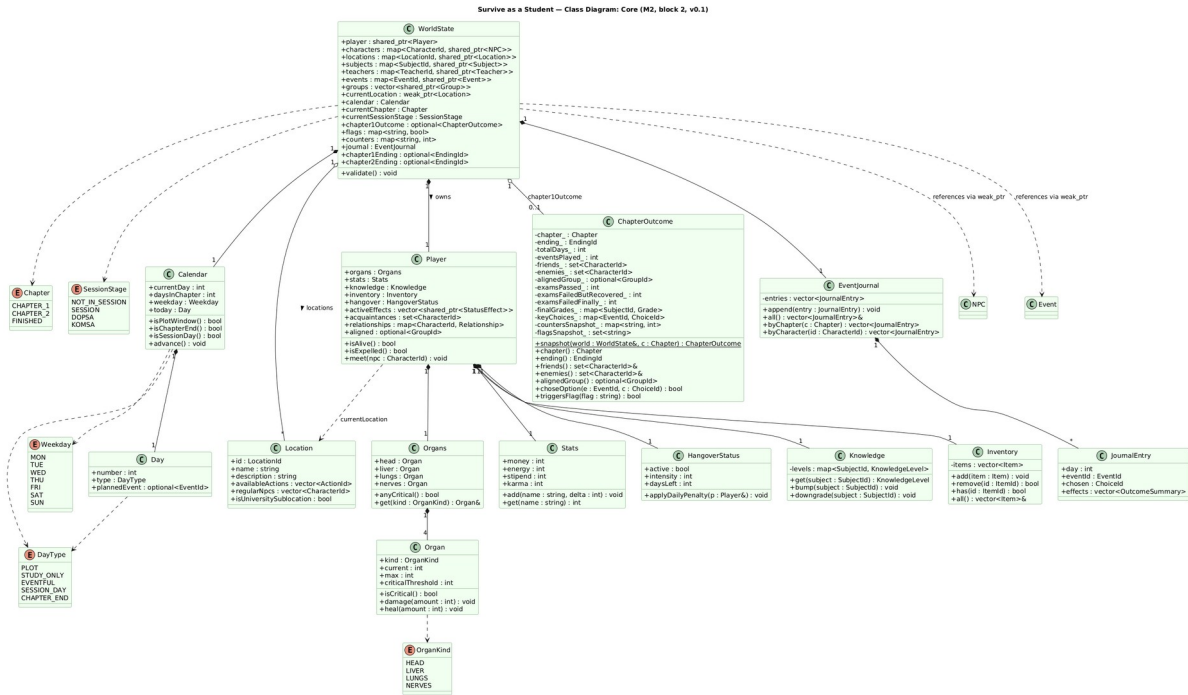


Рис. 2. Класс-диаграмма: ядро (WorldState, Player, Calendar)

4. Персонажи и социальная подсистема

4.1. Классификация NPC

В игре присутствуют два типа NPC: важные (сюжетные) и фоновые. Важные NPC имеют имя, идеологию, расписание (на каких локациях бывают в какие дни недели), диалоговый ключ для подгрузки реплик, флаг `isImportant = true`. Фоновые NPC — это `BackgroundNPC`, наследующийся от NPC с минимумом полей: имя, текущая локация, случайная «флейворная» реплика. Их назначение — оживлять локации, не влияя на сюжет.

4.2. Идеологические группы

В игре три стабильные идеологические группы: Димона, Артёма и Саши. Каждая имеет лидера (NPC), членов (другие NPC) и собственную `Ideology` (основная ось — `CONFORMIST / REBEL / NEUTRAL`, плюс интенсивность убеждений). Игрок может присоединиться к одной из групп (поле `Player::aligned`), остаться нейтральным или демонстративно выйти из всех.

Каждая группа имеет агрегированное отношение к игроку (`Group::aggregatedRelation`), которое пересчитывается по правилам: отношение группы равно, грубо говоря, усреднённое отношение её членов, с поправкой на конфликт идеологий.

4.3. Relationship

Отношения игрока с конкретным NPC хранятся как структура `Relationship`: численное значение в диапазоне $-100..+100$, статус-проекция (`HATE / COLD / NEUTRAL / FRIENDLY / CLOSE`) и лог изменений для отладки. Пороги предлагаются такими: `friend` при $\geq +40$, `enemy` при ≤ -40 . Точные значения — открытый вопрос, зависят от балансировки и могут корректироваться.

4.4. Расписание и случайные встречи

Каждый важный NPC имеет расписание вида `std::map<Weekday, LocationId>`. `EventSystem`, подбирая событие на день с типом `EVENTFUL`, с вероятностью около 25% инициирует случайную встречу: выбирает NPC, для которого сегодняшний день недели в расписании совпадает с локацией, где сейчас игрок. Это обеспечивает «случайные» эпизоды вроде: «по дороге в универ встречаешь Димона после вчерашней пьянки». Полная структура социальной подсистемы показана на рис. 3.

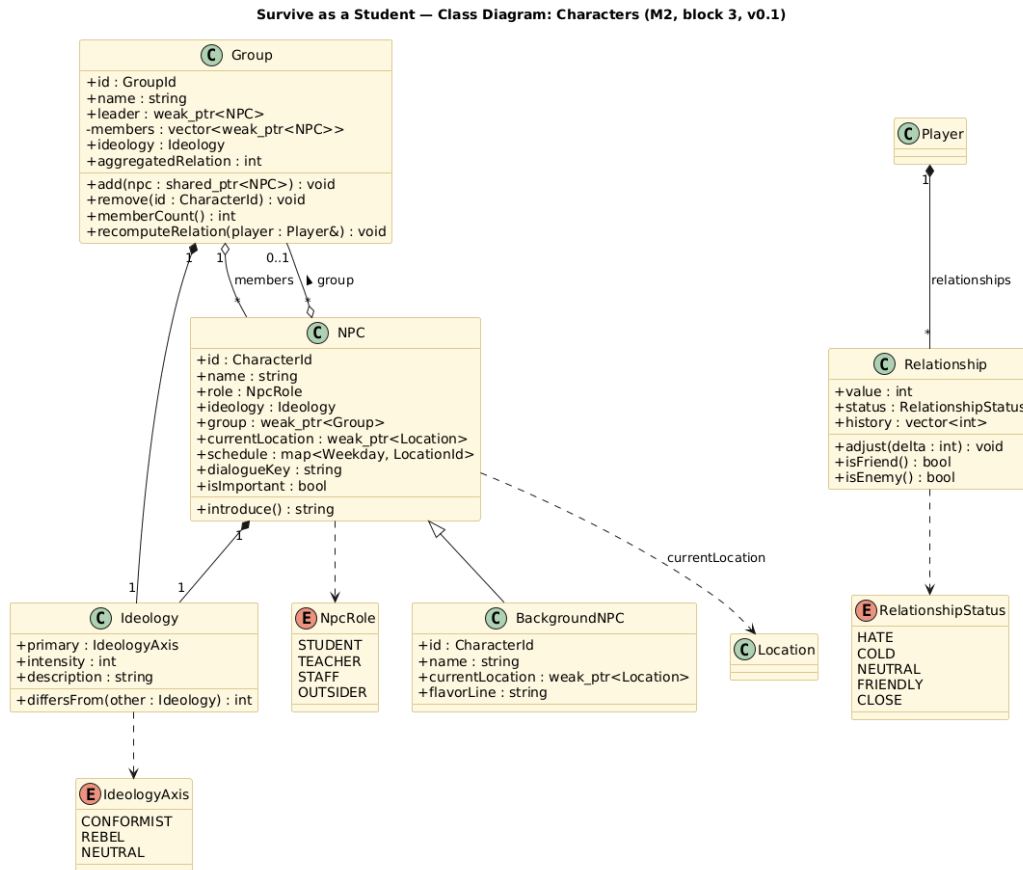


Рис. 3. Класс-диаграмма: персонажи и отношения

5. События и выборы

5.1. Модель события

Событие (Event) — шаблон, загруженный из JSON. Его структура:

- `id` — уникальный строковый идентификатор.
- `kind` — тип события: `PLOT`, `SIDE`, `RANDOM_ENCOUNTER`, `SESSION`, `TEST`, `PARTY`, `ROUTINE`.
- `title`, `description` — заголовок и описание для отображения.
- `trigger` — объект `Trigger`, определяющий условия появления события.
- `choices` — вектор вариантов выбора. Если вариантов меньше двух, событие становится информационным.
- `chapter` — к какой главе привязано событие.
- `location` — опциональная привязка к локации.
- `participants` — слабые ссылки на NPC, участвующих в событии.

5.2. Триггеры

`Trigger` — условие, при котором событие попадает в кандидаты на показ. Перечисление `TriggerKind`: `DAY_NUMBER`, `DAY_TYPE`, `LOCATION`, `FLAG`, `RELATIONSHIP`, `RANDOM`, `COMPOSITE`. Параметры конкретного триггера хранятся в `map<string, string>`; их валидация и интерпретация — задача метода `Trigger::matches(WorldState&)`.

Композитный триггер (`COMPOSITE`) позволяет собирать логические выражения вида «(день $\geq$ 5 AND локация == бар) OR флаг «вписался_к_артёму» == true». Для первой версии достаточно двухуровневой логики; при необходимости на М3 язык условий будет расширен.

5.3. Выбор и последствие

`Choice` — один вариант, отображаемый игроку. У него есть текст, список `Requirement` (условия видимости и активности), и один `Outcome` — последствие. Методы `Choice::visibleIf()` и `Choice::enabledIf()` определяют, виден ли вариант и можно ли его выбрать.

`Outcome` — декларативное описание последствия. Перечисление `OutcomeKind`: `STAT_DELTA`, `ORGAN_DELTA`, `RELATIONSHIP_DELTA`, `FLAG_SET`, `COUNTER_INC`, `ITEM_GIVE`, `ITEM_REMOVE`, `KNOWLEDGE_CHANGE`, `END_CHAPTER`, `GAME_OVER`. Параметры — `map<string, string>`. Метод `Outcome::apply(WorldState&)` — единственная точка, где событие мутирует мир. Ни `Choice`, ни `Event` сами мир не меняют.

Если один выбор должен произвести несколько эффектов, используется Outcome с kind = COMPOSITE и списком подэффектов в параметрах. Это позволяет оставить модель «один Choice → один Outcome» и не усложнять сигнатуры.

6. Академическая подсистема

6.1. Предметы и преподаватели

Subject представляет учебный предмет: id, имя, слабая ссылка на преподавателя, список тем, семестр (1 или 2). Teacher — отдельный класс, технически ссылающийся на NPC (npcRef), но содержащий академические параметры: strictness (строгость оценки) и cheatTolerance (шанс поймать на шпоре).

6.2. Вопросы

Question — элемент банка вопросов: subject, topic, текст, правильный ответ, массив отвлекающих ответов, difficulty (1–5). Difficulty влияет на то, какой уровень знания темы (KnowledgeLevel: KNOWN, PARTIAL, UNKNOWN) требуется для уверенного ответа.

6.3. Шпоры

CheatSheet — предмет инвентаря, привязанный к конкретному предмету и теме. У него два ключевых параметра: successRate (вероятность, что шпора действительно даст правильный ответ, если ей удалось воспользоваться) и detectionRisk (базовый шанс, что преподаватель заметит). Метод CheatSheet::use(Player, Teacher) возвращает true при успешном применении и учитывает Teacher::cheatTolerance.

Если преподаватель ловит студента на шпоре, последствия серьёзные: вопрос засчитывается как неверный, отношение с преподавателем снижается на значительную величину, а по итогам экзамена возможно автоматическое «не зачёт».

6.4. Экзамен

Exam — объект, представляющий один конкретный заход к преподавателю (сессия, допса или комса). Поля: subject, teacher, пул вопросов, questionsPerAttestation (рабочее: 3 на сессии, 5 на допсе, 7 на комсе), passingGrade (рабочее: C), тип аттестации (AttestationKind: SESSION, DOPSA, KOMSA).

Метод Exam::run(WorldState&) реализует весь поток экзамена: выбор вопросов из пула, опрос по каждому вопросу с учётом знаний игрока, возможное применение шпор, учёт строгости преподавателя и выставление итоговой оценки (Grade: F, D, C, B, A, EXCELLENT). Возвращает Grade.

6.5. Каскад провалов

Если студент не сдаёт сессию, его ждёт допса (дополнительная сдача). Если проваливает допсу — комса (комиссионная сдача). Если провалит и комсу — отчисление и плохая концовка главы (BAD_EXPELLED). Каскад реализован в AcademicSystem: методы onSessionFailed / onDopsaFailed / onKomsaFailed меняют sessionStage и, если нужно, вызывают EndingSystem.

Связи между подсистемами событий и академии (в частности, через CheatSheet, Subject и общий enum OutcomeKind) показаны на рис. 4.

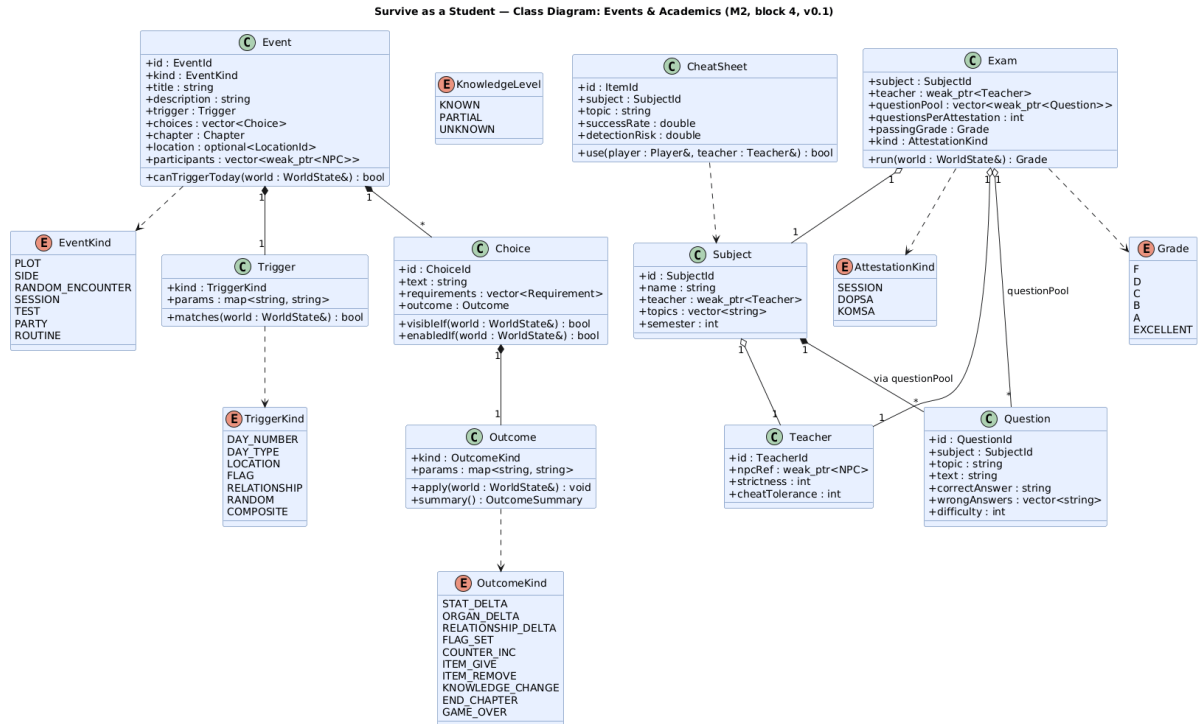


Рис. 4. Класс-диаграмма: события и академическая подсистема

7. Системы-«глаголы»

7.1. Список систем

Игровая логика разнесена по семи системам. Ни одна из них не хранит собственного состояния — только ссылку на `WorldState`, которая передаётся в конструкторе. Каждая система — `stateless` с точки зрения данных; её методы принимают и возвращают информацию, а все изменения фиксирует в `WorldState`.

7.2. EventSystem

Центральная система игрового цикла. Методы: `pickEventForToday()` подбирает шаблон события по `Trigger.matches`; `rollRandomEncounter()` с настраиваемой вероятностью инициирует случайную встречу; `presentChoices(Event&)` отображает варианты игроку; `applyChoice(Event&, ChoiceId)` применяет последствия; `recordToJournal(Event&, ChoiceId)` пишет запись в `EventJournal`.

7.3. StatusEffectSystem

Отвечает за здоровье и статусы. Метод `tickDaily()` вызывается каждый день — снижает интенсивность бодуна, восстанавливает энергию. Методы `applyHangover(int intensity, int days)`, `organDamage(OrganKind, int)`, `organHeal(OrganKind, int)`, `checkCriticalOrgans()` — точки входа для других систем и для `Outcome::apply`.

7.4. RelationshipSystem

Операции над отношениями: `adjust(CharacterId, int delta)` — точечное изменение; `recomputeGroupRelations()` — полный пересчёт агрегированных отношений групп; `triggerIdeologyConflict(CharacterId)` — применить штраф за идеологическое расхождение; хуки `onPartyAttendance(Event&)`, `onGroupAlignment(GroupId)` — точки входа для конкретных игровых событий.

7.5. AcademicSystem

Фасад для учёбы: `studyTopic(SubjectId, string topic)` повышает уровень знания темы; `takeExam(SubjectId, AttestationKind)` запускает `Exam::run`; `onSessionFailed/onDopsaFailed/onKomsaFailed` — каскадные реакции на провал; `cheatAttempt(CheatSheet, Teacher)` — обёртка над `CheatSheet::use` с обработкой обнаружения.

7.6. EconomySystem

Работа с деньгами: `payStipend()` (раз в условный период), `spend(int amount, string reason)` с возвратом `false` при недостатке средств, `earn(int, string)`, `canAfford(int)`. Названия причин (`reason`) пишутся в журнал — для отладки и постигровой аналитики.

7.7. TimeSystem

Единственный владелец метода `advanceDay()`. Внутри: вызов `StatusEffectSystem::tickDaily()`, продвижение `Calendar`, проверка `isChapterEnd`, передача управления `EventSystem::pickEventForToday` для обработки обычного дня. Также методы `enterSession/enterDopsa/enterKomsa/exitSession` управляют `sessionStage`, `transitionToNextChapter` переводит партию из главы 1 в главу 2.

7.8. EndingSystem

Отвечает за финалы глав. `checkBadEndingTriggers()` проверяется в начале каждого дня и возвращает `EndingId`, если сработал плохой триггер (критический орган, отчисление, арест, банкротство во второй главе). `computeChapterEnding(Chapter)` вызывается на `CHAPTER_END` и определяет итоговый финал (`CH*_GOOD`, если не было плохих триггеров, иначе — соответствующая плохая концовка). `finalize(Chapter)` создаёт и возвращает `ChapterOutcome` через статический метод `snapshot`.

Состав семи систем, их методы и связи между ними представлены на рис. 5.

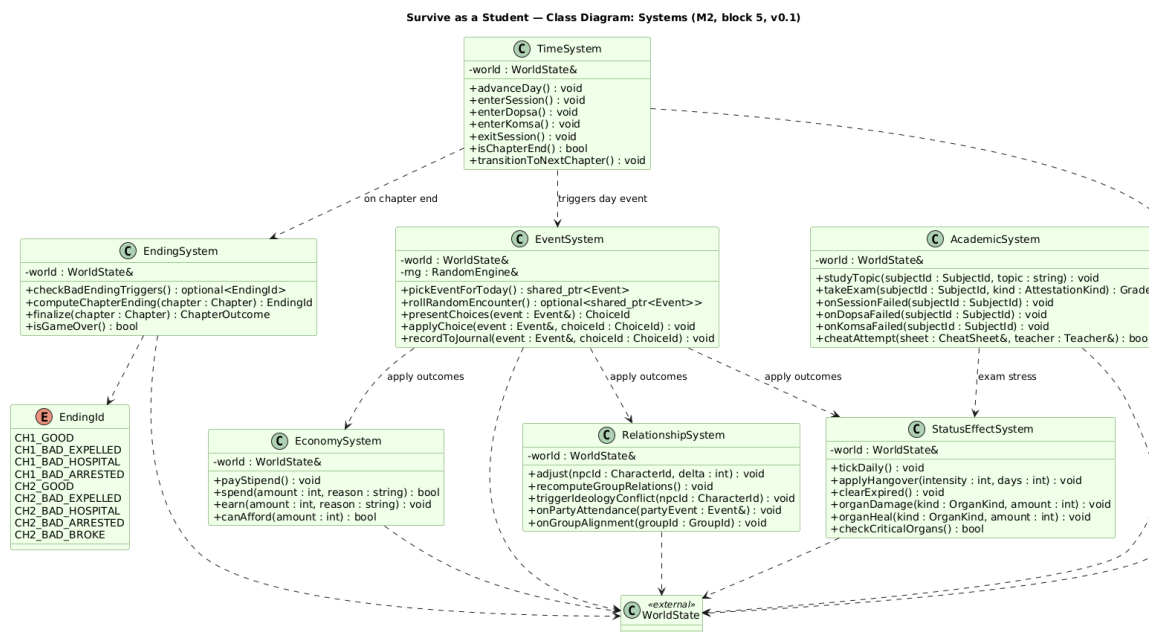


Рис. 5. Класс-диаграмма: системы-«глаголы»

8. Главный игровой цикл

8.1. Общая структура

Весь игровой процесс укладывается в единый repeat-цикл по дням. Цикл прерывается только в двух случаях: наступил плохой финал (критический орган, отчисление, арест) либо завершилась вторая глава (любой концовкой).

8.2. Обработка одного дня

Один проход цикла — один игровой день. Последовательность шагов:

13. `TimeSystem.advanceDay()` продвигает календарь.
14. `StatusEffectSystem.tickDaily()` обновляет статусы игрока.
15. `EndingSystem.checkBadEndingTriggers()` — если сработало, переход к финализации главы.
16. Разбор дня по `Calendar.today.type`: `SESSION_DAY` запускает `AcademicSystem.takeExam` для каждого экзамена; `CHAPTER_END` запускает `EndingSystem.computeChapterEnding` и `finalize`; обычный день (`PLOT / STUDY_ONLY / EVENTFUL`) идёт через `EventSystem.pickEventForToday`.
17. Если подобрано событие — `EventSystem.presentChoices`, получение выбора игрока, `EventSystem.applyChoice`, `EventSystem.recordToJournal`. Если событий нет — игрок выбирает свободное действие (учёба, сон, прогулка, магазин).
18. `GameApp` рендерит сводку дня.

8.3. Почему так

Ключевые принципы цикла: (1) только `TimeSystem.advanceDay()` двигает календарь, чтобы не было багов с «двойным днём»; (2) проверка плохих триггеров стоит до обработки дня, чтобы умерший персонаж не получил сначала обычный день и только потом финал; (3) idle-дни существуют: не каждый день обязан быть событием, и это нормально для дней 11–45, которые моделируют учебную рутину.

Полная диаграмма активности главного цикла с ветвлениями по типу дня и обработкой перехода между главами показана на рис. 6.

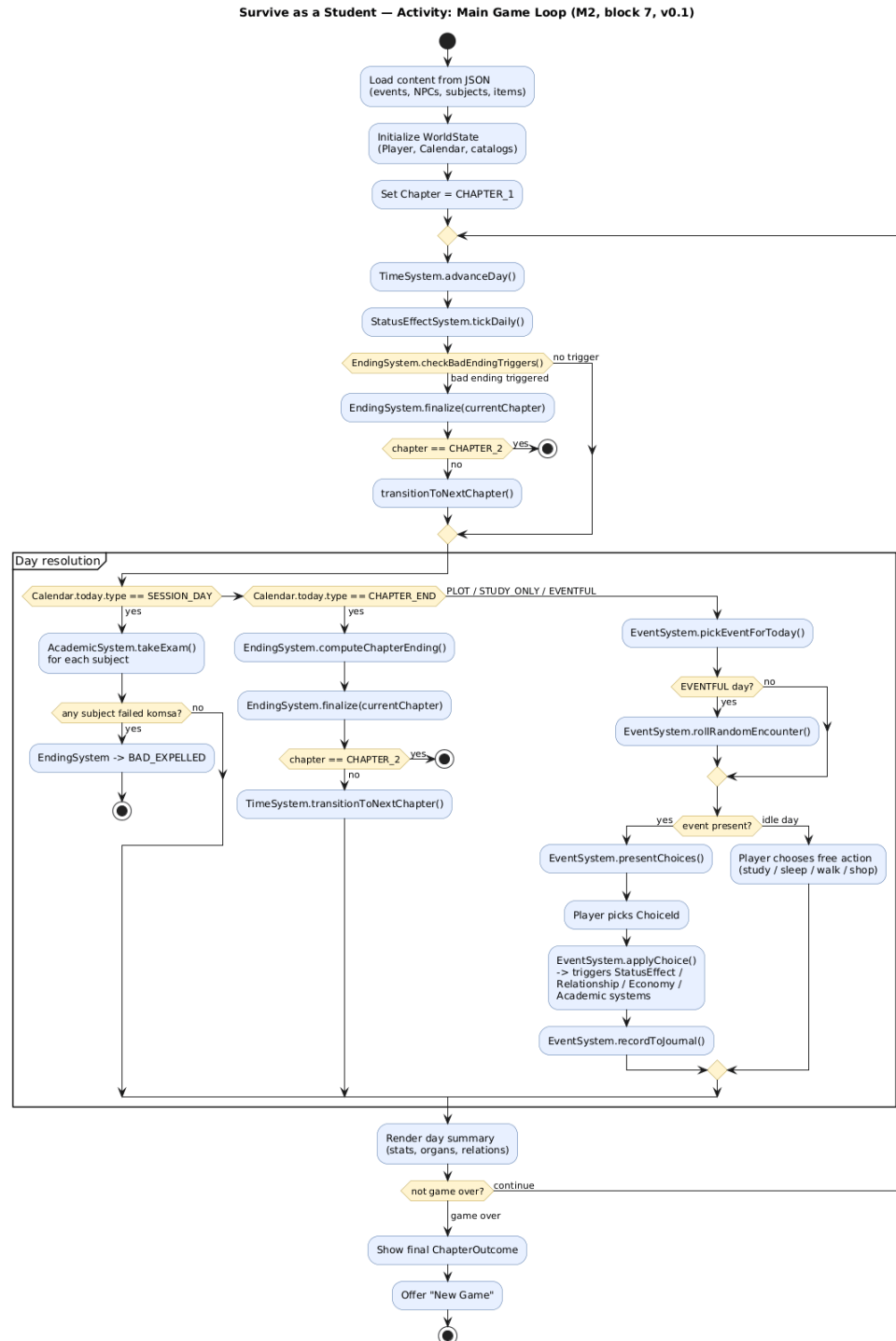


Рис. 6. Activity-диаграмма: главный игровой цикл

9. Ключевые сценарии

9.1. Обработка события с выбором

Сценарий демонстрирует, как один игровой день с событием проходит через все слои. Порядок вызовов:

19. Игрок инициирует переход на следующий день → `GameApp` → `TimeSystem.advanceDay()`.
20. `TimeSystem.advanceDay()` вызывает `StatusEffectSystem.tickDaily()` (снижение похмелья, регенерация энергии).
21. `TimeSystem` вызывает `EventSystem.pickEventForToday()`. Тот перебирает `WorldState::events` и вызывает `Event.canTriggerToday` для каждого, возвращая первое подходящее.
22. `EventSystem.presentChoices(Event&)` фильтрует варианты через `visibleIf` и `enabledIf`, отображает оставшиеся игроку.
23. Игрок выбирает вариант, возвращается `ChoiceId`.
24. `EventSystem.applyChoice(Event&, ChoiceId)` достаёт соответствующий `Outcome` и вызывает `Outcome.apply(world)`.
25. Внутри `Outcome.apply` — ветвление по `kind`: `STAT_DELTA` → `EconomySystem` (если деньги) или `Stats.add`; `ORGAN_DELTA` → `StatusEffectSystem.organDamage/Heal`; `RELATIONSHIP_DELTA` → `RelationshipSystem.adjust`; `FLAG_SET / COUNTER_INC` → `WorldState` напрямую.
26. `EventSystem.recordToJournal` создаёт запись в `EventJournal` — уже с актуальными последствиями.
27. Управление возвращается в `GameApp`, тот рендерит сводку дня.

Ключевое наблюдение: `Outcome.apply` — единственная точка мутации `WorldState` в контексте события. Ни `Event`, ни `Choice` сами состояние не меняют; они только описывают, что могло бы произойти. Подробная последовательность вызовов представлена на рис. 7.

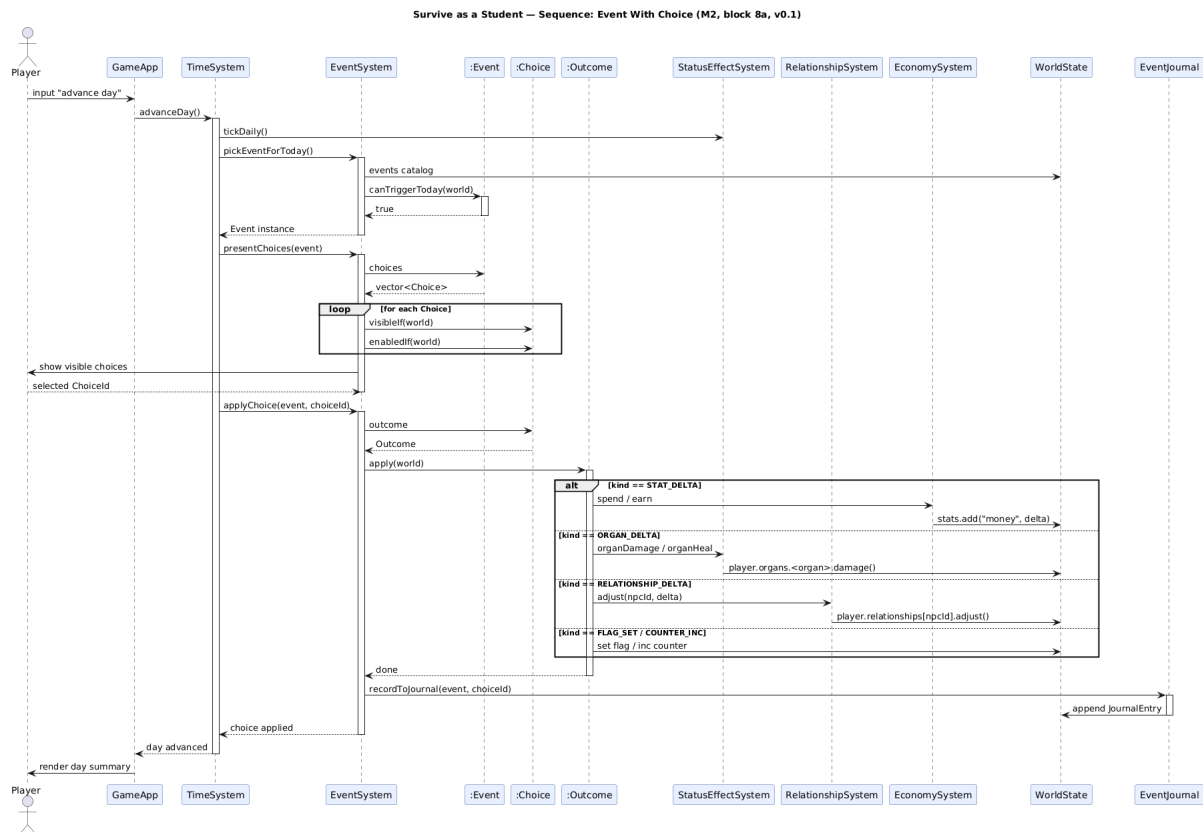


Рис. 7. Sequence-диаграмма: обработка события с выбором

9.2. Проведение экзамена

Сценарий описывает, как работает один экзамен на SESSION_DAY. Порядок:

28. TimeSystem.advanceDay() видит, что сегодня SESSION_DAY. Для каждого предмета в текущей сессии вызывается AcademicSystem.takeExam(SubjectId, AttestationKind).
29. AcademicSystem достаёт шаблон Exam из WorldState, вызывает Exam.run(world).
30. Exam выбирает N вопросов из пула (questionsPerAttestation — 3/5/7 в зависимости от вида аттестации).
31. По каждому вопросу Exam смотрит Player.knowledge[subject][topic]: KNOWN — правильный ответ; PARTIAL — взвешенный бросок монетки с учётом difficulty; UNKNOWN — игрок может применить шпору либо получает неверный ответ.
32. При использовании шпору CheatSheet.use(Player, Teacher) проверяет detectionRisk против Teacher.cheatTolerance. Поимка — штраф к отношению с преподавом и засчёт вопроса как неверного; иначе — бросок на successRate.
33. После всех вопросов Exam учитывает Teacher.strictness и выставляет Grade.
34. Если Grade $\geq$ passingGrade — PASS, результат пишется в историю экзаменов; иначе FAIL.

35. При FAIL AcademicSystem триггерит каскад: SESSION → enterDopsa; DOPSA → enterKomsa; KOMSA → EndingSystem.checkBadEndingTriggers, который вернёт BAD_EXPELLED.

36. После обработки всех экзаменов дня GameApp рендерит сводку сессии.

Вся случайность экзамена локализована внутри Exam.run и использует единый RandomEngine из слоя Engine. Никакого rand() в других местах — это обеспечивает, например, возможность в будущем посеивать RNG для детерминированных тестов или replays. Полный поток экзамена с вложенными loop-блоками по предметам и вопросам, а также с каскадом SESSION → DOPSA → KOMSA → BAD_EXPELLED, представлен на рис. 8.

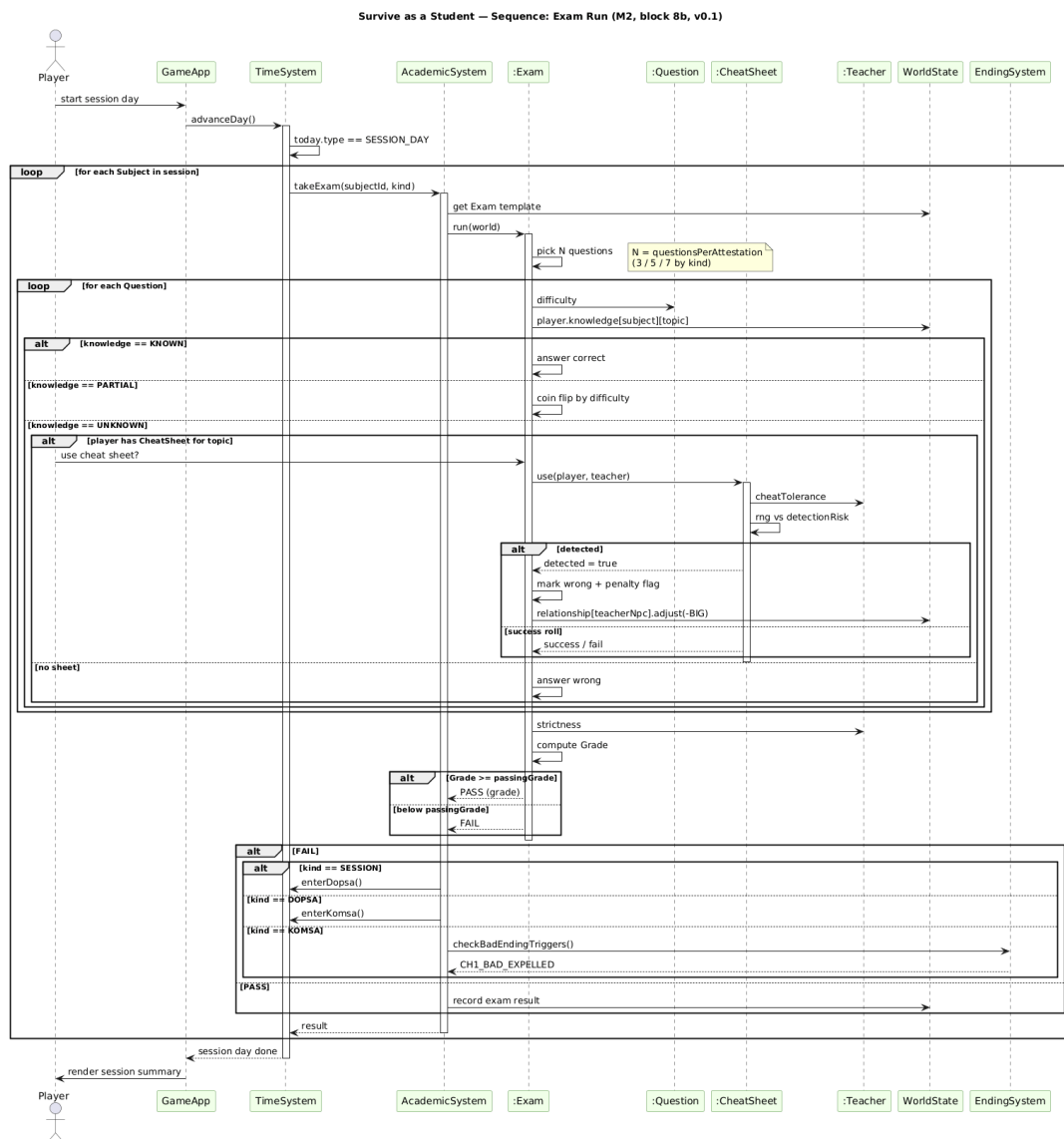


Рис. 8. Sequence-диаграмма: проведение экзамена

10. Инварианты и обработка ошибок

10.1. Проверки на старте

При загрузке контента AssetLoader проверяет корректность JSON-структуры на уровне схемы: все обязательные поля присутствуют, типы совпадают, идентификаторы уникальны. При ошибке — немедленный отказ с информативным сообщением. Подход fail-fast: некорректный контент — это ошибка сценариста, её надо показывать на этапе сборки, а не во время игры.

10.2. Проверки во время игры

WorldState.validate() вызывается (в debug-сборке) после каждой системной операции, модифицирующей мир. В release-сборке — только при старте и при переходе между главами, чтобы не просаживать производительность. Метод проверяет весь список инвариантов, приведённый в разделе 3.3.

Все weak_ptr внутри доменных объектов перед использованием приводятся через lock(); если ссылка обнулилась — это нарушение инварианта и повод для исключения. Такой подход исключает классы ошибок с висячими указателями и делает причину сбоя явной.

10.3. Сбои в событиях

Если Outcome ссылается на несуществующий stat или organ, ошибка фиксируется в журнале, ход игрока откатывается, появляется сообщение об ошибке контента. Это защита от опечаток в JSON; в debug-сборке может быть усилена до fail-fast.